UTM

UNIVERSITI TEKNOLOGI MALAYSIA

SECP3623

Database Programming

Assignment 1

Name	Matric No
AHMAD ZIYAAD BIN MOHD ABBAS	A23CS0206
DAMIYA AINA BINTI BASIR ABD SHAMMAD	A23CS0220

1. Database Creation Command Execution

SQL COMMAND:

```
-- =========================
-- TASK 1: DATABASE CREATION
-- =========================
DROP DATABASE IF EXISTS hostel_mgmt_yad;
CREATE DATABASE hostel_mgmt_yad;
USE hostel_mgmt_yad;
```

OUTPUT:

Explanation:

- Deletes the old database (if it exists).
- Creates a new empty database.
- Activates it for subsequent queries.

2. CREATE TABLE: room_types

SQL COMMAND:

```
-- ROOM TYPES
CREATE TABLE room_types (
    type_id INT AUTO_INCREMENT PRIMARY KEY,
    type_name VARCHAR(50) NOT NULL UNIQUE,
    rent DECIMAL(8,2) NOT NULL,
    deposit DECIMAL(8,2) NOT NULL,
    capacity INT NOT NULL
);
```

OUTPUT:

Explanation:

- Stores room categories.
  Includes: rent, deposit, capacity, unique type_name.

3. CREATE TABLE: rooms

SQL COMMAND:

```
-- ROOMS
CREATE TABLE rooms (
    room_id INT AUTO_INCREMENT PRIMARY KEY,
    type_id INT NOT NULL,
    room_no VARCHAR(10) NOT NULL UNIQUE,
    floor_no INT NOT NULL,
    is_occupied BOOLEAN DEFAULT FALSE,
    FOREIGN KEY (type_id) REFERENCES room_types(type_id)
);
```

OUTPUT:

Explanation:

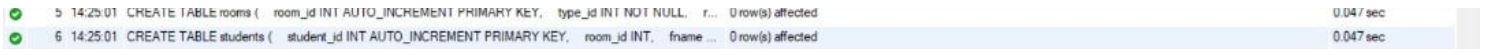
- Stores physical rooms.
  type_id is FK → must exist in room_types.
  is_occupied defaults to FALSE.

4. CREATE TABLE: students

SQL COMMAND:

```
-- STUDENTS
CREATE TABLE students (
    student_id INT AUTO_INCREMENT PRIMARY KEY,
    room_id INT,
    fname VARCHAR(50) NOT NULL,
    lname VARCHAR(50),
    status ENUM('ACTIVE', 'NON_ACTIVE') NOT NULL,
    checkin_date DATE,
    email VARCHAR(100) UNIQUE,
    FOREIGN KEY (room_id) REFERENCES rooms(room_id)
);
```

OUTPUT:



Explanation:

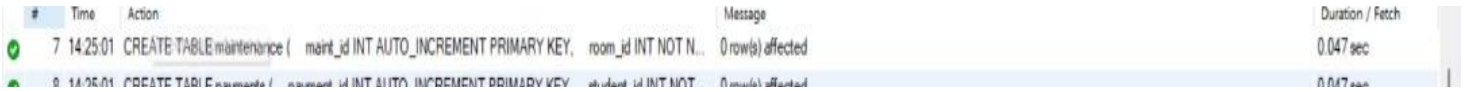
- Stores student details, room assignment, check-in date, status, and unique email.

5. CREATE TABLE: maintenance

SQL COMMAND:

```
-- MAINTENANCE
CREATE TABLE maintenance (
    maint_id INT AUTO_INCREMENT PRIMARY KEY,
    room_id INT NOT NULL,
    issue_desc VARCHAR(255) NOT NULL,
    severity ENUM('LOW', 'MEDIUM', 'HIGH') NOT NULL,
    status ENUM('OPEN', 'RESOLVED') NOT NULL,
    reported_on DATE NOT NULL,
    resolved_on DATE,
    FOREIGN KEY (room_id) REFERENCES rooms(room_id)
);
```

OUTPUT:



Explanation:

- Stores maintenance issues per room.
- Includes severity, status (OPEN/RESOLVED), and dates.

6. CREATE TABLE: payments

SQL COMMAND:

```
-- PAYMENTS
CREATE TABLE payments (
    payment_id INT AUTO_INCREMENT PRIMARY KEY,
    student_id INT NOT NULL,
    amount DECIMAL(8,2) NOT NULL,
    paid_on DATE NOT NULL,
    method ENUM('CASH', 'FPX', 'CARD', 'TNG') NOT NULL,
    note VARCHAR(100),
    FOREIGN KEY (student_id) REFERENCES students(student_id)
);
```

OUTPUT:

Explanation:

- Stores rent payment history.
  method is restricted to CASH, FPX, CARD, TNG.

7. ALTER TABLE add email2

SQL COMMAND:

```
-- Add a unique email column (if not added above)
ALTER TABLE students ADD COLUMN email2 VARCHAR(100) UNIQUE;
```

OUTPUT:

Explanation:

Adds a second email to students, also unique.

8. Temporary Table Creation & Deletion

SQL COMMAND:

```
-- Create and safely drop a temporary test table
CREATE TABLE temp_table (test_id INT);
DROP TABLE temp_table;
```

OUTPUT:

Explanation:

Shows your ability to create and remove test tables.

9. INSERT Data for All Tables

SQL COMMAND:

```
-- 1. DATA INSERTION
INSERT INTO room_types (type_name, rent, deposit, capacity) VALUES
('Single', 600.00, 300.00, 1),
('Double', 500.00, 250.00, 2),
('Premium', 850.00, 400.00, 1),
('Family', 1000.00, 500.00, 4),
('Economy', 400.00, 200.00, 2),
('Compact', 450.00, 250.00, 2),
('Executive', 950.00, 450.00, 1),
('Deluxe', 700.00, 350.00, 2),
('Suite', 1200.00, 600.00, 3),
('Student', 550.00, 300.00, 2);

INSERT INTO rooms (type_id, room_no, floor_no, is_occupied) VALUES
(1,'A101',1,FALSE),(2,'A102',1,FALSE),(3,'A103',1,FALSE),(4,'B201',2,FALSE),
(5,'B202',2,FALSE),(6,'C301',3,FALSE),(7,'C302',3,FALSE),(8,'A104',1,FALSE),
(9,'B203',2,FALSE),(10,'C303',3,FALSE);
```

```
INSERT INTO students (room_id,fname,lname,status,checkin_date,email) VALUES
(1,'Ahmad','Ziyaad','ACTIVE','2025-10-10','ahmadz@utm.my'),
(2,'Ali','Morrisane','ACTIVE','2025-10-11','alim@utm.my'),
(3,'Sara','Parker','NON_ACTIVE','2025-09-15','sarap@utm.my'),
(4,'Nur','Haliza','ACTIVE','2025-10-20','nurh@utm.my'),
(5,'Daniel','Tee','ACTIVE','2025-10-25','danielt@utm.my'),
(6,'Aiman','Bana','NON_ACTIVE','2025-08-05','aimanb@utm.my'),
(7,'Faiz','Shukri','ACTIVE','2025-10-30','faizshuk@utm.my'),
(8,'Tun','Razak','ACTIVE','2025-10-28','tunrazak@utm.my'),
(9,'Donald','Trump','NON_ACTIVE','2025-07-22','donaldt@utm.my'),
(10,'Mokhtar','Dahari','ACTIVE','2025-10-15','mokhdah@utm.my');

INSERT INTO maintenance (room_id,issue_desc,severity,status,reported_on,resolved_on) VALUES
(1,'Water leak in bathroom','MEDIUM','OPEN','2025-10-20',NULL),
(2,'Broken chair','LOW','RESOLVED','2025-09-15','2025-09-20'),
(3,'Fan not working','HIGH','OPEN','2025-10-25',NULL),
(4,'Window crack','LOW','RESOLVED','2025-08-10','2025-08-15'),
(5,'Aircond noisy','MEDIUM','OPEN','2025-10-26',NULL),
(6,'Power socket fault','HIGH','OPEN','2025-10-28',NULL),
(7,'Light bulb fused','LOW','RESOLVED','2025-08-01','2025-08-02'),
(8,'Door lock issue','MEDIUM','OPEN','2025-10-29',NULL),
(9,'Desk damage','LOW','OPEN','2025-10-27',NULL),
(10,'Ceiling leak','HIGH','RESOLVED','2025-08-15','2025-08-20');
```

```
INSERT INTO payments (student_id,amount,paid_on,method,note) VALUES
(1,500.00,'2025-10-01','CASH','October Rent'),
(2,600.00,'2025-10-02','FPX','October Rent'),
(4,1000.00,'2025-11-01','CARD','November Rent'),
(5,800.00,'2025-11-02','TNG','November Rent'),
(7,950.00,'2025-10-01','CARD','October Rent'),
(8,700.00,'2025-11-03','FPX','November Rent'),
(10,550.00,'2025-11-05','CASH','November Rent'),
(3,500.00,'2025-09-01','TNG','September Rent'),
(6,850.00,'2025-08-01','FPX','August Rent'),
(9,600.00,'2025-07-01','CARD','July Rent');
```

OUTPUT:

Explanation:

This avoids orphan foreign key issues and ensures proper referential integrity.

10. UPDATE Occupancy Status

SQL COMMAND:

```
UPDATE rooms
SET is_occupied = TRUE
WHERE room_id IN (
    SELECT DISTINCT room_id FROM students WHERE status='ACTIVE'
);
```

OUTPUT:

Explanation:

- Marks a room as occupied if it has at least 1 ACTIVE student.

11.DELETE Maintenance Records

SQL COMMAND:

```
DELETE FROM maintenance
WHERE status='RESOLVED' AND reported_on < DATE_SUB(CURDATE(), INTERVAL 60 DAY);
```

OUTPUT:

Explanation:

Removes maintenance entries that:

- Are RESOLVED
- And reported more than 60 days ago from today

12. Filtering Query using BETWEEN

SQL COMMAND:

```
-- BETWEEN
SELECT * FROM room_types WHERE rent BETWEEN 400 AND 800;
```

OUTPUT:

	Time	Action	Message	Duration / Fetch
19	14:25:01	SELECT * FROM room_types WHERE rent BETWEEN 400 AND 800 LIMIT 0, 1000	6 row(s) returned	0.000 sec / 0.000 sec

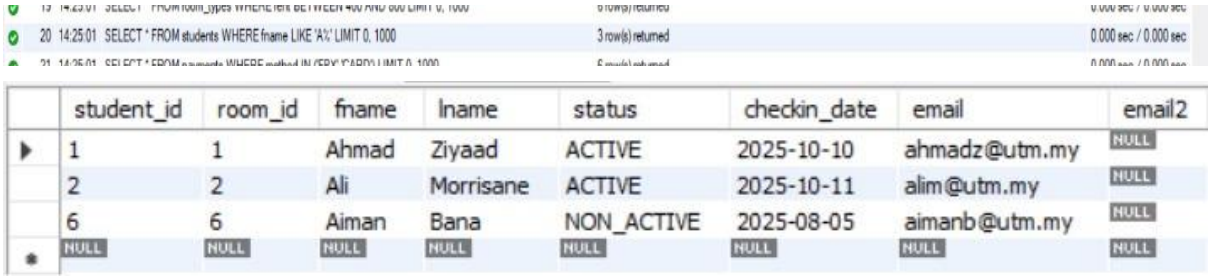
	type_id	type_name	rent	deposit	capacity
▶	1	Single	600.00	300.00	1
	2	Double	500.00	250.00	2
	5	Economy	400.00	200.00	2
	6	Compact	450.00	250.00	2
	8	Deluxe	700.00	350.00	2
	10	Student	550.00	300.00	2
*	NULL	NULL	NULL	NULL	NULL

13. Filtering Query using LIKE

SQL COMMAND:

```
-- LIKE
SELECT * FROM students WHERE fname LIKE 'A%';
```

OUTPUT:



	Time	Action	Message	Duration / Fetch
20	14:25:01	SELECT * FROM students WHERE fname LIKE 'A%' LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec

	student_id	room_id	fname	lname	status	checkin_date	email	email2
▶	1	1	Ahmad	Ziyaad	ACTIVE	2025-10-10	ahmadz@utm.my	NULL
	2	2	Ali	Morrisane	ACTIVE	2025-10-11	alim@utm.my	NULL
	6	6	Aiman	Bana	NON_ACTIVE	2025-08-05	aimanb@utm.my	NULL
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

14. Filtering Query using IN

SQL COMMAND:

```
-- IN
SELECT * FROM payments WHERE method IN ('FPX','CARD');
```

OUTPUT:

payment_id	student_id	amount	paid_on	method	note
2	2	600.00	2025-10-02	FPX	October Rent
3	4	1000.00	2025-11-01	CARD	November Rent
5	7	950.00	2025-10-01	CARD	October Rent
6	8	700.00	2025-11-03	FPX	November Rent
9	6	850.00	2025-08-01	FPX	August Rent
10	9	600.00	2025-07-01	CARD	July Rent
NULL	NULL	NULL	NULL	NULL	NULL

15. Filtering Query using AND / OR / NOT

SQL COMMAND:

```
-- AND / OR / NOT
SELECT * FROM rooms
WHERE (floor_no = 1 OR floor_no = 2) AND NOT is_occupied = FALSE;
```

OUTPUT:

room_id	type_id	room_no	floor_no	is_occupied
1	1	A101	1	1
2	2	A102	1	1
4	4	B201	2	1
5	5	B202	2	1
8	8	A104	1	1
NULL	NULL	NULL	NULL	NULL

Explanation

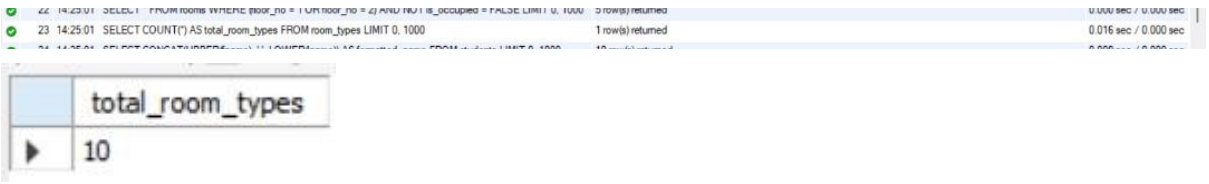
- Must be on floor 1 or 2
- Must be TRUE (occupied)

16. COUNT Function

SQL COMMAND:

```
-- Aggregate
SELECT COUNT(*) AS total_room_types FROM room_types;
```

OUTPUT:



	total_room_types
▶	10

17. String Formatting Functions

SQL COMMAND:

```
-- String Functions
SELECT CONCAT(UPPER(fname), ' ', LOWER(lname)) AS formatted_name FROM students;
```

OUTPUT:

	formatted_name
▶	AHMAD ziyaad
	ALI morrisane
	SARA parker
	NUR haliza
	DANIEL tee
	AIMAN bana
	FAIZ shukri
	TUN razak
	DONALD trump
	MOKHTAR dahari

18. AVG and ROUND Function

SQL COMMAND:

```
-- AVG and ROUND example
SELECT ROUND(AVG(rent),2) AS avg_rent FROM room_types;
```

OUTPUT:

	avg_rent
▶	720.00

19. VIEW Creation

SQL COMMAND:

```
-- 1. CREATE VIEW
CREATE OR REPLACE VIEW v_room_status AS
SELECT
    r.room_no,
    t.type_name,
    t.rent,
    r.floor_no,
    t.capacity,
```

Explanation:

CREATE OR REPLACE VIEW v_room_status AS

- This creates a view named v_room_status.
- A view is like a virtual table — it does not store data itself, it just stores a query you can run like a table.
- OR REPLACE ensures that if a view with the same name exists, it will be replaced.

SELECT r.room_no, t.type_name, t.rent, r.floor_no, t.capacity

- These are basic columns selected from the rooms and room_types tables.
- room_no → the room number.
- type_name → the type of room (Single, Double, etc.).
- rent → the rent for that type.
- floor_no → which floor the room is on.
- capacity → how many students can stay in that room.

20. Total Number of Students per Room Type

SQL COMMAND:

```
-- a) Total number of students per room type
SELECT t.type_name, COUNT(s.student_id) AS total_students
FROM room_types t
JOIN rooms r ON t.type_id = r.type_id
JOIN students s ON r.room_id = s.room_id
GROUP BY t.type_name;
```

OUTPUT:

	type_name	total_students
▶	Compact	1
	Deluxe	1
	Double	1
	Economy	1
	Executive	1
	Family	1
	Premium	1
	Single	1
	Student	1
	Suite	1

21. Average Rent & Total Deposit per Room Type

SQL COMMAND:

```
-- b) Average rent and total deposit per room type
SELECT type_name,
       ROUND(AVG(rent),2) AS avg_rent,
       ROUND(SUM(deposit),2) AS total_deposit
FROM room_types
GROUP BY type_name;
```

OUTPUT:

	type_name	avg_rent	total_deposit
▶	Compact	450.00	250.00
	Deluxe	700.00	350.00
	Double	500.00	250.00
	Economy	400.00	200.00
	Executive	950.00	450.00
	Family	1000.00	500.00
	Premium	850.00	400.00
	Single	600.00	300.00
	Student	550.00	300.00
	Suite	1200.00	600.00

22. Monthly Payment Totals (Year & Month)

SQL COMMAND:

```
-- c) Monthly payment totals grouped by year & month
SELECT YEAR(paid_on) AS yr, MONTH(paid_on) AS mn, SUM(amount) AS total
FROM payments
GROUP BY yr, mn;
```

OUTPUT:

	yr	mn	total
▶	2025	10	2050.00
	2025	11	3050.00
	2025	9	500.00
	2025	8	850.00
	2025	7	600.00

23. Count of OPEN Maintenance Issues per Floor (Using HAVING)

SQL COMMAND:

```
-- d) Count of OPEN maintenance issues per floor with HAVING
SELECT r.floor_no, COUNT(m.maint_id) AS open_issues
FROM maintenance m
JOIN rooms r ON m.room_id = r.room_id
WHERE m.status='OPEN'
GROUP BY r.floor_no
HAVING COUNT(m.maint_id) > 2;
```

OUTPUT:

	floor_no	open_issues
▶	1	3

24. Categorize Rent Levels (LOW, MEDIUM, HIGH)

SQL COMMAND:

```
-- 3. CASE FUNCTION TO CATEGORIZE RENT LEVELS
SELECT type_name, rent,
CASE
    WHEN rent < 600 THEN 'LOW'
    WHEN rent BETWEEN 600 AND 900 THEN 'MEDIUM'
    ELSE 'HIGH'
END AS rent_category
FROM room_types;
```

OUTPUT:

31 14:25:01 SELECT type_name, rent, CASE WHEN rent < 600 THEN 'LOW' WHEN rent BETWEEN 600 AND 900 ... 10 row(s) returned 0.000 sec / 0.000 sec

	type_name	rent	rent_category
▶	Single	600.00	MEDIUM
	Double	500.00	LOW
	Premium	850.00	MEDIUM
	Family	1000.00	HIGH
	Economy	400.00	LOW
	Compact	450.00	LOW
	Executive	950.00	HIGH
	Deluxe	700.00	MEDIUM
	Suite	1200.00	HIGH
	Student	550.00	LOW

A short summary table listing all created tables and the number of records per table

Table name	records
room_types	10
rooms	10
students	10
maintanence	6
payments	10

Explanation:

- room_types → 10 rows inserted.
- rooms → 10 rows inserted.
- students → 10 rows inserted.
- maintenance → 10 rows inserted, 4 rows deleted (resolved > 60 days old), so 6 remaining.
- payments → 10 rows inserted.